



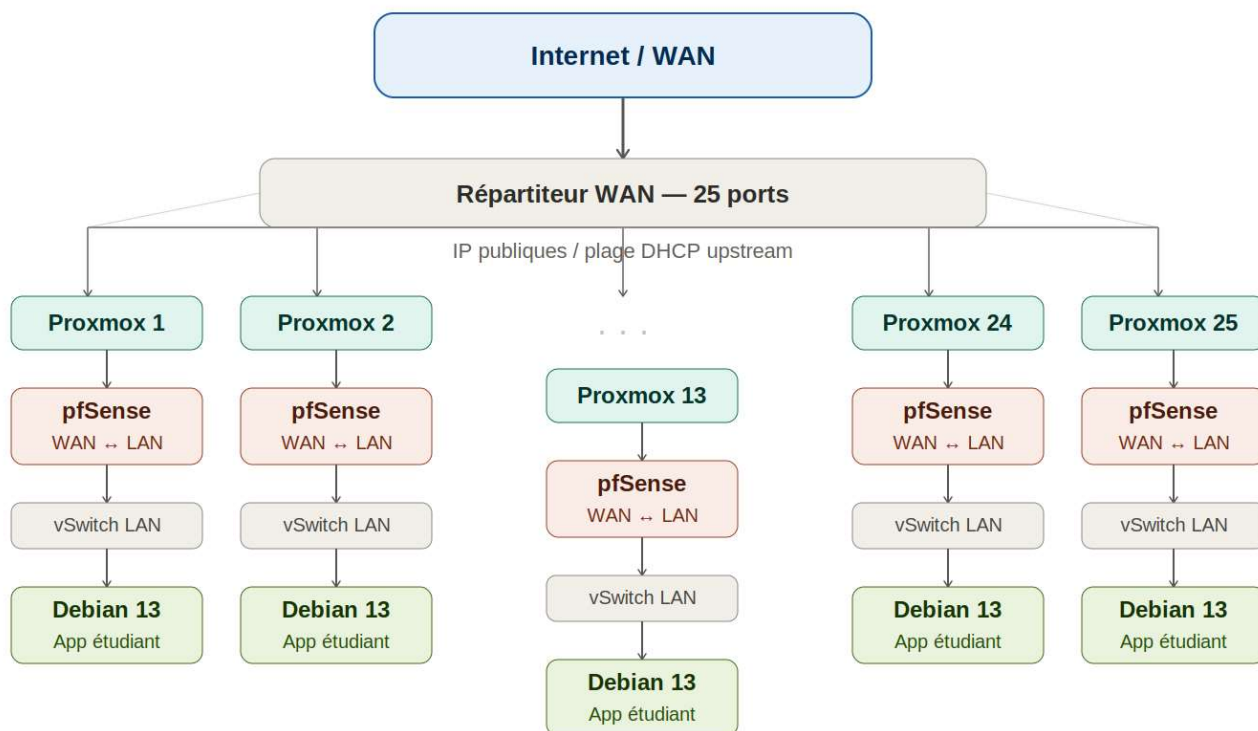
Pipeline CI/CD sans Docker

Déploiement d'applications étudiants via Dokploy

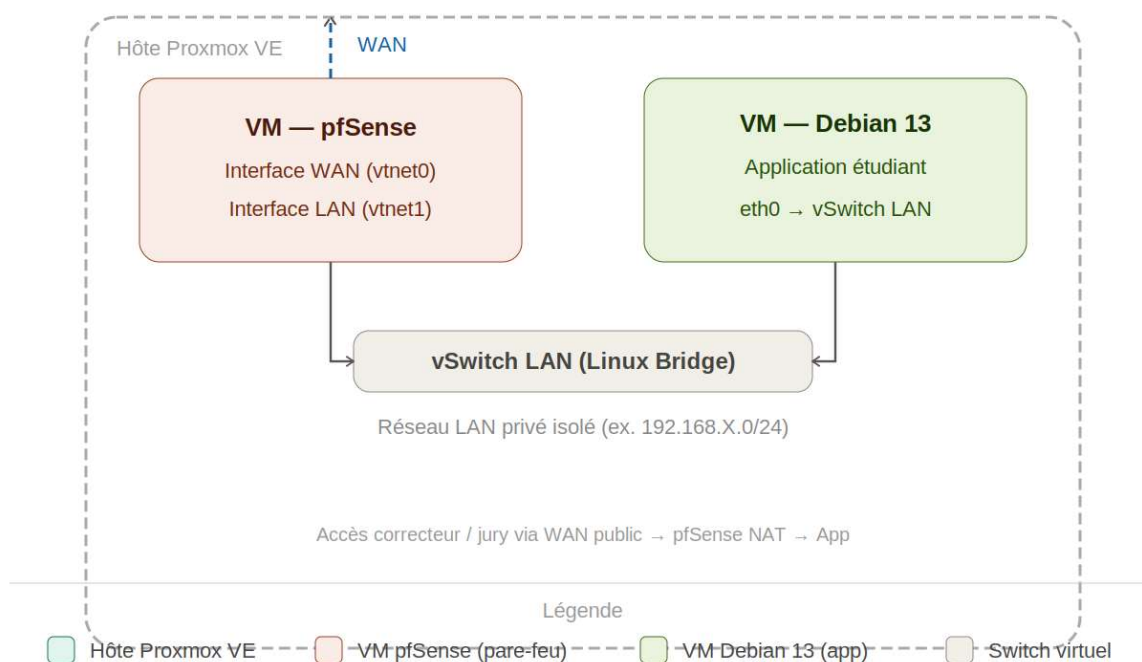
Proxmox VE · pfSense · Debian 13 · Épreuve E6

Ce guide pas-à-pas explique comment installer et configurer Dokploy sur chaque VM Debian 13 d'une instance Proxmox, permettant aux étudiants de déployer leur application web via un pipeline CI/CD Git-natif, sans Docker.

Architecture de déploiement — 25 instances Proxmox VE



Détail — architecture interne d'un nœud (exemple : Proxmox 1)



0. Prérequis et architecture

Architecture du déploiement

Chaque étudiant dispose de :

- 1 hôte Proxmox VE (donné par l'examineur)
- 1 VM pfSense : pare-feu WAN/LAN + NAT + règles de port-forward
- 1 VM Debian 13 : serveur applicatif avec Dokploy installé

- Un dépôt Git (GitHub, GitLab ou Gitea local)

Prérequis système (VM Debian 13)

Composant	Exigence minimale
OS	Debian 13 (Trixie) 64-bit
RAM	2 Go minimum (4 Go recommandés)
Disque	20 Go minimum
CPU	2 vCPU
Réseau	Interface LAN connectée au vSwitch pfSense
Accès	Sudo ou root, SSH activé
Git	Version 2.x ou supérieure
Node.js	v18+ (pour Dokploy)

Étape 1 — Installation de Dokploy sur Debian 13

1.1 Mise à jour initiale du système

Connectez-vous en SSH à votre VM Debian 13, puis exécutez :

```
# Connexion SSH depuis votre machine
ssh etudiant@192.168.X.10

# Mise à jour des paquets
sudo apt update && sudo apt upgrade -y

# Outils de base
sudo apt install -y curl wget git unzip ufw
```

1.2 Installation de Node.js (v20 LTS)

```
# Ajout du dépôt NodeSource
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

# Installation de Node.js
sudo apt install -y nodejs

# Vérification
node -v      # doit afficher v20.x.x
npm -v      # doit afficher 10.x.x
```

1.3 Installation de Dokploy

Dokploy fournit un script d'installation officiel. Il installe l'application, configure le service systemd, et ouvre l'interface web.

```
# Téléchargement et exécution du script officiel
curl -sSL https://dokploy.com/install.sh | sudo bash
```

```
# L'installation prend 2 à 5 minutes
# À la fin, vous verrez :
#   Dokploy is running on http://0.0.0.0:3000
```

i Info Le script installe Dokploy en mode « standalone » (sans Docker). Il crée un utilisateur système dédié et un service systemd appelé dokploy.

1.4 Vérification du service

```
# Statut du service
sudo systemctl status dokploy

# Activer au démarrage
sudo systemctl enable dokploy

# Voir les logs en temps réel
sudo journalctl -u dokploy -f
```

Étape 2 — Configuration pfSense (NAT / Port-Forward)

Pour accéder à l'interface Dokploy depuis l'extérieur (jury, correcteur), il faut rediriger le port 3000 de l'IP WAN vers la VM Debian.

2.1 Règle de NAT (port-forward)

Champ	Valeur
Interface	WAN
Protocole	TCP
Port source (ext)	3000
IP destination	192.168.X.10 (IP LAN de la VM Debian)
Port destination	3000
Description	Dokploy UI — étudiant

Chemin dans pfSense : Firewall → NAT → Port Forward → Add

💡 Astuce Optionnel : exposer également le port 80/443 de l'application une fois déployée.

Étape 3 — Première connexion à Dokploy

3.1 Accès à l'interface

Depuis votre navigateur (ou celui du correcteur) :

```
# Depuis le réseau local (LAN)
http://192.168.X.10:3000

# Depuis l'extérieur via l'IP WAN pfSense
http://<IP-WAN-pfSense>:3000
```

3.2 Création du compte administrateur

Au premier lancement, Dokploy affiche un formulaire de création du compte admin :

1. Saisir un e-mail valide (ex. : `etudiantX@synapsure.local`)
2. Définir un mot de passe fort (≥ 8 caractères)
3. Cliquer sur « Create Account »
4. Se connecter avec les identifiants créés

⚠ Attention Conservez bien vos identifiants. Dokploy ne propose pas de réinitialisation de mot de passe en mode standalone sans configuration SMTP préalable.

Étape 4 — Création du projet et du service

4.1 Nouveau projet

5. Dans le tableau de bord, cliquer sur « Create Project »
6. Nom du projet : `app-e6-etudiantX`
7. Description : Application E6 — [Nom Prénom]
8. Valider avec « Create »

4.2 Ajout d'un service « Application »

9. Dans le projet, cliquer sur « + Add Service » → « Application »
10. Nom du service : `web`
11. Sélectionner le type : « Git » (GitHub / GitLab / dépôt privé)
12. Valider

Étape 5 — Configuration du dépôt Git

5.1 Connexion au dépôt

Dans l'onglet « General » du service, renseigner :

Champ	Valeur
Repository URL	<code>https://github.com/etudiant/app-e6.git</code>
Branch	<code>main</code> (ou <code>master</code>)
Build Path	<code>/</code> (racine du projet)
Build Type	Nixpacks (détection automatique)

Port exposé	8080 (ou le port de votre app)
--------------------	--------------------------------

5.2 Clé SSH de déploiement (dépôt privé)

Si le dépôt est privé, générer une clé SSH sur la VM et l'ajouter comme « Deploy Key » sur GitHub/GitLab :

```
# Génération de la clé (sur la VM Debian)
ssh-keygen -t ed25519 -C "dokploy-deploy" -f ~/.ssh/dokploy_key -N ""

# Afficher la clé publique à copier dans GitHub/GitLab
cat ~/.ssh/dokploy_key.pub
```

Ajouter cette clé publique dans : Settings → Deploy Keys → Add deploy key (GitHub) ou Settings → Repository → Deploy Keys (GitLab).

Étape 6 — Configuration du build (Nixpacks)

6.1 Pourquoi Nixpacks ?

Dokploy utilise Nixpacks comme moteur de build par défaut. Nixpacks détecte automatiquement le langage et les dépendances de votre projet (Python, Node.js, PHP, Ruby, Java...) et génère un environnement de build reproductible, sans nécessiter de Dockerfile.

6.2 Exemples de détection automatique

Stack	Fichier détecté	Commande de démarrage
Node.js	package.json	npm start
Python	requirements.txt	python app.py
PHP	composer.json / index.php	php -S 0.0.0.0:8080
Java	pom.xml	java -jar target/*.jar

6.3 Variables d'environnement

Dans l'onglet « Environment » du service Dokploy, définir les variables nécessaires (sans les mettre dans le dépôt Git) :

```
# Exemples de variables d'environnement
PORT=8080
DATABASE_URL=postgresql://user:pass@localhost:5432/mydb
NODE_ENV=production
SECRET_KEY=votre_clef_secrete
```



Sécurité Ne jamais stocker de secrets (mots de passe, clés API) dans le dépôt Git. Utiliser exclusivement les variables d'environnement de Dokploy.

Étape 7 — Premier déploiement

7.1 Lancer le build et le déploiement

13. Dans le service, aller sur l'onglet « Deployments »
14. Cliquer sur « Deploy » (bouton vert)
15. Surveiller les logs de build en temps réel
16. Attendre le statut « Running » (icône verte)

7.2 Vérifier que l'application tourne

```
# Sur la VM Debian — vérifier le processus
sudo systemctl status dokploy

# Vérifier que le port est écouté
ss -tlnp | grep 8080

# Test rapide en local
curl http://localhost:8080
```

Depuis le navigateur, accéder à l'application via :

```
http://<IP-WAN-pfSense>:8080 # si NAT configuré sur port 8080
```

Étape 8 — Automatisation CI/CD par Webhook Git

8.1 Récupérer l'URL du webhook Dokploy

17. Dans le service, onglet « General »
18. Copier l'URL affichée sous « Webhook URL »

Elle ressemble à :

```
http://192.168.X.10:3000/api/deploy/<token-unique>
```

8.2 Configurer le webhook sur GitHub

19. Aller dans le dépôt GitHub → Settings → Webhooks → Add webhook
20. Payload URL : coller l'URL Dokploy
21. Content type : application/json
22. Secret : laisser vide ou renseigner un token (optionnel)
23. Events : sélectionner « Just the push event »
24. Cliquer « Add webhook »

8.3 Configurer le webhook sur GitLab

25. Aller dans Settings → Webhooks
26. URL : coller l'URL Dokploy
27. Trigger : cocher « Push events »
28. SSL verification : désactiver si HTTP local
29. Cliquer « Add webhook »

8.4 Test du pipeline complet

Sur votre machine de développement :

```
# Modifier un fichier
echo '# Test CI/CD' >> README.md
```

```
# Commit et push
git add README.md
git commit -m "test: pipeline CI/CD Dokploy"
git push origin main

# → Dokploy reçoit le webhook et lance automatiquement
# le rebuild + redéploiement de l'application
```

✅ **Résultat attendu** Dès que le push est effectué, l'onglet Deployments de Dokploy affiche une nouvelle entrée. Le statut passe de « Building » à « Running » en quelques secondes à quelques minutes selon la taille du projet.

Étape 9 — Récapitulatif et checklist de validation

Avant de remettre votre travail au jury, vérifier que chaque point est validé :

☑	Élément à vérifier	Commande de vérification
☐	Dokploy installé et actif	<code>systemctl status dokploy</code>
☐	Interface web accessible sur port 3000	<code>curl localhost:3000</code>
☐	Compte admin créé	Connexion réussie dans le navigateur
☐	Projet et service créés dans Dokploy	Interface Dokploy
☐	Dépôt Git connecté (URL + branche)	Onglet General du service
☐	Premier déploiement réussi (statut Running)	Onglet Deployments
☐	Application accessible depuis le WAN	Navigateur (IP WAN:port)
☐	Webhook Git configuré	GitHub/GitLab Settings
☐	Push déclenche un redéploiement auto	<code>git push</code> → Deployments
☐	Variables d'environnement définies	Onglet Environment

Étape 10 — Dépannage courant

Problèmes fréquents et solutions

Symptôme	Solution
Port 3000 inaccessible	Vérifier les règles pfSense + <code>sudo ufw allow 3000</code>

Build échoué : dep not found	Vérifier package.json ou requirements.txt
Webhook non reçu	Vérifier l'URL du webhook et la connectivité WAN
Statut « Error »	Consulter journalctl -u dokploy --no-pager -n 50
App ne démarre pas	Vérifier la variable PORT et le port exposé
Clé SSH refusée	Vérifier que la clé publique est dans les Deploy Keys du dépôt